

---

# MUTATION TESTING IN JAVA



## | IL PROBLEMA: CODE COVERAGE

80%

Coverage potrebbe non essere sufficiente in alcuni casi

La Code Coverage misura solo quali linee di codice sono state attraversate durante l'esecuzione dei test, non se il test ha effettivamente verificato la correttezza logica.

| **QUIS CUSTODIET IPSOS CUSTODES?**

# CHI CONTROLLA I CONTROLLORI?

## **Unit Tests**

Controllano il Codice di Produzione.



## **Mutation Testing**

Controlla gli Unit Tests.

# | COS'È IL MUTATION TESTING?

"L'inserimento deliberato di difetti artificiali (mutanti) nel codice per verificare se la suite di test è abbastanza robusta da rilevarli."

## 1. Injection

Il tool modifica il bytecode  
(es.  $a + b \rightarrow a - b$ ).

## 2. Execution

Esegue i test contro il codice  
mutato.

## 3. Analysis

Se il test fallisce (Red) =  
Successo.  
Se il test passa (Green) =  
Fallimento.

# | UN PO' DI STORIA

Il Mutation Testing non è una novità. È nato nel mondo accademico negli anni '70, ma è diventato utilizzabile solo recentemente grazie alla potenza di calcolo moderna.

**1971:** Proposto da Richard Lipton (CMU).

**1978:** Paper fondamentale di DeMillo, Lipton & Sayward: *"Hints on Test Data Selection: Help for the Practicing Programmer"*.

**2010s:** Arrivo di tool efficienti come PIT (Java) e Stryker (JS).



Per 40 anni è stato considerato "troppo lento" per l'industria. Oggi è lo standard per i sistemi mission-critical.

# | TEORIA: COMPETENT PROGRAMMER HYPOTHESIS

## L'ipotesi del Programmatore Competente

"I programmatori tendono a scrivere codice che è *quasi* corretto."

---

La maggior parte dei bug non sono errori logici massivi e complessi, ma piccole deviazioni sintattiche dal codice corretto.

- Usare < invece di <=
- Usare && invece di ||
- Dimenticare di chiamare un metodo void.

Per questo motivo, mutare piccoli operatori sintattici è efficace per simulare errori reali.

# | TEORIA: COUPLING EFFECT

## L'Effetto Accoppiamento

"Test data that detects all simple faults is sensitive enough to detect more complex faults."

DeMillo et al. ipotizzarono che i bug complessi sono spesso il risultato dell'interazione di molti bug semplici (accoppiati).

Se la tua suite di test è in grado di uccidere tutti i "mutanti semplici" (1° ordine), statisticamente è molto probabile che rilevi anche bug architetturali più complessi.

# | TERMINOLOGIA ESSENZIALE

## **Mutant**

La versione modificata del codice.

## **Killed Mutant**

BENE. Il test è fallito. Il mutante è stato rilevato e "ucciso".

## **Survived Mutant**

MALE. Il test è passato (verde) nonostante il codice fosse rotto. Il mutante è "sopravvissuto".

## **Equivalent Mutant**

Falso positivo. Una modifica sintattica che non cambia la logica.

# | IL MUTATION SCORE

La metrica definitiva per la qualità dei test.

$$\text{Score} = \text{Killed} / ( \text{Total} - \text{Equivalent} ) * 100$$

## Code Coverage vs Mutation Score

Puoi avere 100% Code Coverage e 0% Mutation Score (se non hai asserzioni).

Non puoi avere un alto Mutation Score senza un'alta Code Coverage.

## Obiettivo Realistico

Puntare al 100% è spesso anti-economico a causa dei mutanti equivalenti.

Un punteggio di 80-90% è eccellente per progetti Java enterprise.

# | OPERATORI DI MUTAZIONE

Pitest (il tool standard per Java) usa diversi "Mutators".

| Categoria     | Esempio                                              |
|---------------|------------------------------------------------------|
| Math          | + → -, * → /, i++ → i--                              |
| Conditionals  | > → >=, == → !=                                      |
| Return Values | return true → return false, return obj → return null |
| Void Calls    | Rimuove chiamate a metodi void (es. list.add())      |

# MATH MUTATOR

```
public int calculateTotal(int price, int tax) {  
    // MUTANT: return price - tax;  
    return price + tax;  
}
```

**Perché  
sopravvive?**

Se il tuo test case usa  $\text{tax} = 0$ , il risultato di  $+$  e  $-$  è identico.

Fix: Usa valori non neutri (es. price 10, tax 2).

```
@Test  
void testTotal() {  
    // 10 + 0 = 10  
    // 10 - 0 = 10 → SURVIVED  
    assertEquals(10, svc.calculateTotal(10, 0));  
}
```

# | CONDITIONAL BOUNDARY

## Il classico "Off-by-one"

Il mutante cambia il confine inclusivo in esclusivo.

Se testi solo age = 20 (Adulto) e age = 10 (Bambino), entrambi i percorsi funzionano sia col codice originale che col mutante.

Fix: Devi testare esattamente il confine (Edge Case).

`assert isAdult(18) == true`

```
public boolean isAdult(int age) {  
    // MUTANT: if (age > 18) → if (age >= 18)  
    if (age >= 18) {  
        return true;  
    }  
    return false;  
}
```

# | STRONG VS WEAK MUTATION

## Weak Mutation

Il mutante causa uno stato interno errato immediatamente dopo l'esecuzione dell'istruzione mutata.

*Condizione necessaria ma non sufficiente per uccidere il mutante.*

## Strong Mutation

Lo stato interno errato si propaga fino a causare un output osservabile errato o un crash.

*Questo è ciò che conta per i test.*

# | PIT (PITEST)

# PIT

State of the Art for JVM

PIT è lo standard de facto. Prima di PIT, i tool come Jester ricompilavano il codice per ogni mutante (lentissimo).

- ✓ Manipolazione Bytecode (veloce)
- ✓ Supporto JUnit 4, 5, TestNG, Spock
- ✓ Plugin Maven & Gradle

# | SETUP MAVEN RAPIDO

```
org.pitest    pitest-maven      1.15.0
org.pitest    pitest-junit5-plugin 1.2.0
com.myapp.core.*
```

```
mvn org.pitest:pitest-maven:mutationCoverage
```

# | PIT ARCHITECTURE

Come fa ad essere veloce?

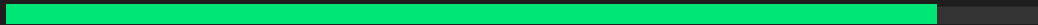
1. Scan Classpath
2. Run Tests (Coverage)
3. Generate Mutants (RAM)
4. Run ONLY covering tests per mutant

PIT sa quali test toccano quali righe (Coverage). Se un mutante è alla riga 50, PIT esegue solo i test che passano per la riga 50.

# | LEGGERE IL REPORT

## Line Coverage

Quanto codice è stato eseguito.



## Mutation Coverage

Quanto codice è stato testato robustamente.



La differenza tra le due barre è il tuo "Gap di Qualità" (Assertion Gap).

# | IL PROBLEMA DELLE PERFORMANCE

$$\text{Tempo} = O(\text{Test} \times \text{Mutanti})$$

Esempio reale: Un progetto con 1000 test che impiega 1 minuto.

Genera 5000 mutanti. Tempo stimato: 5000 minuti (3 giorni)?

Grazie all'ottimizzazione (esegue solo test rilevanti), magari scende a 30 minuti. Ma è comunque troppo per ogni commit.

# | OTTIMIZZAZIONE 1: INCREMENTAL ANALYSIS

La "Killer Feature" di PIT per la CI.

PIT salva un file di hash dei risultati precedenti.

Alla run successiva, riesegue i mutanti SOLO SE:

- 1 La classe mutata è cambiata.
- 2 Il test che copriva quella classe è cambiato.

Risultato: Run di secondi invece di minuti.

# | OTTIMIZZAZIONE 2: SCOPE

## Multithreading

Usa tutti i core della CPU.

Attenzione ai test non thread-safe!

## Filtering

Ignora Config, UI, Generated Code (Lombok). Muta solo la business logic.

## Features

+auto\_threads

Lascia che PIT decida il numero ottimale di thread.

# | MUTANTI EQUIVALENTI

```
// CODE
int i = 0;
while (i < 10) {
    ...
    i++;
}

// MUTANT
while (i != 10) {
    ...
    i++;
}
```

Nessun test può ucciderlo. PIT lo segna come "Survived", abbassando il tuo score ingiustamente.

# | GESTIRE I MUTANTI EQUIVALENTI

Non c'è un modo automatico perfetto (Problema dell'arresto).

## Default Mutators

PIT usa di default un set di operatori ("DEFAULTS") che minimizza la generazione di mutanti equivalenti.

## Espansione Codice

A volte rifattorizzare il codice per renderlo più semplice elimina l'equivalenza.

## Ignore / Filter

Configura PIT per ignorare certe linee o metodi problematici (es. `hashCode()`, `toString()`).

# | IL PROBLEMA DEI LOOP INFINITI

```
for (int i = 0; i < 10; i++)  
// MUTANT: rimozione di i++  
for (int i = 0; i < 10; )
```

Alcuni mutanti creano loop infiniti. PIT ha un meccanismo di Timeout.

Se un test impiega troppo tempo (più del normale + margine), PIT lo uccide e considera il mutante come "TIMED\_OUT" (che conta come Killed).

Attenzione: Rallenta molto l'analisi.

# | CONFIGURAZIONE AVANZATA: GRUPPI

Puoi scegliere quanto essere aggressivo.

## DEFAULTS

Bilanciato. Pochi falsi positivi.  
Consigliato per iniziare.

## STRONGER

Aggiunge mutazioni più aggressive (es. rimozione costanti). Più falsi positivi.

## ALL

Tutto. Preparati a gestire migliaia di mutanti equivalenti.

# | ALTERNATIVE A PIT

| Tool   | Pro                               | Contro                             |
|--------|-----------------------------------|------------------------------------|
| PIT    | Standard, Veloce, Attivo          | Bytecode based (a volte complesso) |
| Major  | Integrato nel compilatore (javac) | Meno facile da integrare in Maven  |
| Jumble | Pioniere                          | Non più mantenuto                  |

# BEST PRACTICES DI ADOZIONE

## The Ratchet Effect

In un progetto legacy, non puoi fixare 5000 mutanti. Blocca lo score attuale e impedischi che scenda (il "cricchetto").

## Focus on Logic

Escludi package di infrastruttura. Muta solo dove c'è logica di business complessa (Algoritmi, Calcoli, Validazioni).

## Education

Usa il Mutation Testing per insegnare ai Junior Dev come scrivere test migliori (Assertion Driven).

## Non usarlo sempre

Non eseguirlo in locale ad ogni "Save". Eseguilo prima del Push o nel server CI.

# | IMPATTO PSICOLOGICO

## La falsa sicurezza del Verde

Vedere una barra verde al 100% di coverage dà una scarica di dopamina pericolosa se i test sono vuoti. Il Mutation Testing è un "Reality Check" brutale ma necessario.

### Confidence

Quando il Mutation Score è alto, puoi rifattorizzare il codice con vera tranquillità, sapendo che se rompi qualcosa, i test lo vedranno.

# CONCLUSIONE

"Il Code Coverage ti dice cosa non hai testato.

Il Mutation Testing ti dice quanto bene hai testato ciò che hai testato."

## +Vantaggi

Trova test "Assertion Free"

Migliora drasticamente la qualità dei test

## — Costi

Tempo di esecuzione (CPU intensivo)

Analisi dei Mutanti Equivalenti